

Accessibility UI Tester - User Guide

Jaygovindhan Neelananarayanan

September 2026

Accessibility UI Tester

About

What it does

Requirements

Installation

Repository structure

Quick start

Module guides

Known limitations

Non-URP projects

Running the tests

License

Accessibility UI Tester

A Unity Editor extension for testing and improving the accessibility of in-game UI: colour vision deficiency (CVD) simulation, WCAG contrast checking, automated accessibility reporting, and font-scaling overlap detection.

Built for Unity 6 (URP), with graceful degradation for non-URP projects.

About

This tool was built as the practical output of an MSc Individual Project (Computer Games Technology with VR, City St George's, University of London), looking at whether real-time, in-Editor accessibility tooling can help developers catch colour-contrast and colour-vision-deficiency issues in their UI before shipping. It was evaluated against two independent third-party Unity projects (BayatGames/RedRunner and UnityTechnologies/open-project-1 "Chop Chop"), in addition to internal testing.

What it does

Four independent modules, usable together or separately:

Module	What it does	Requires
Shader Controller	Simulates Protanopia, Deuteranopia, and Tritanopia in real time via a URP Renderer Feature, plus a low-vision blur effect	URP
Contrast Meter	Click any UI element in Play Mode to see its live WCAG contrast ratio, correctly handling semi-transparent/overlapping layers	Play Mode
Report Writer	Scans an entire scene's UI for contrast and font-size violations, exports a JSON report	none
Font Scaler	Globally scales UI text and highlights any elements that start overlapping as a result	none

Requirements

- Unity 6000.0.x LTS or later (built and tested on 6000.0.81f1)
- **Universal Render Pipeline (URP)**, required only for Shader Controller (CVD simulation and blur). Contrast Meter, Report Writer, and Font Scaler work in any render pipeline, including Built-in RP.
- TextMeshPro (Unity's standard package), used for font-size and overlap detection alongside legacy `Text` support.

If your project doesn't have URP installed, or your URP Renderer Feature isn't set up, the tool detects this automatically and hides the Shader Controller section of its window rather than throwing errors. See [Non-URP projects](#) below.

Installation

Option A: Import the package (recommended for most users)

1. Download `AccessibilityUITester.unitypackage` from this repository.
2. In your Unity project: **Assets -> Import Package -> Custom Package...**, select the downloaded file.
3. Keep everything checked, click **Import**.
4. Open the tool: **Window -> Accessibility UI Tester**.

Option B: Clone the repository (if you want the source, tests, or want to contribute)

```
git clone https://github.com/jaygoneelz/  
AccessibilityUITester.git
```

Copy `Assets/_AccessibilityTester/Runtime/` and `Assets/_AccessibilityTester/Editor/` into your own project's `Assets/` folder. The `Tests/` and `TestAssets/` folders are development-only and not needed for normal use; see [Repository structure](#).

Repository structure

Everything lives under `Assets/_AccessibilityTester/` , split into two folders that map to two separate Assembly Definitions:

```
_AccessibilityTester/  
  Runtime/  
    ContrastMeter/      WCAG math + live click-checking  
    FontScaler/         text scaling + overlap detection  
    ReportWriter/       scene scanning + JSON reports  
    ShaderController/   CVD sim + blur (URP-only, see below)  
    Shaders/           CVD + Gaussian blur shaders  
    ScriptableObjects/  DefaultAccessibilityThresholds.asset  
    LICENSE.txt         bundled with the .unitypackage  
  Editor/  
    Core/               AccessibilityTesterWindow,  
  CoreManager  
    ContrastMeter/ FontScaler/ ReportWriter/ ShaderController/  
  Tests/               EditMode + PlayMode NUnit tests  
  TestAssets/          materials for the dev test scene
```

`Runtime/` has no dependency on `UnityEditor` , so none of it gets stripped from a player build. This is where the actual logic lives, one folder per module. `Editor/` is the EditorWindow itself plus a small bridge class per module that wires the Runtime logic up to buttons and fields in the tool window. `Tests/` and `TestAssets/` only matter if you clone the repo; they're left out of the exported `.unitypackage` on purpose, since there's no reason to ship test scaffolding to someone who just wants the tool.

Quick start

1. **Window -> Accessibility UI Tester** to open the tool.
2. If your project uses URP and you want CVD/blur simulation, drag your project's **Universal Renderer Data** asset into the **URP Renderer Data** field (not the URP Pipeline Asset itself; see [Finding your Renderer Data asset](#)).
3. Everything else works immediately. No further setup is needed for Contrast Meter, Report Writer, or Font Scaler.

Module guides

Shader Controller

1. Assign your **Universal Renderer Data** asset (see below) into the tool window.

2. The first time you do this for a given Renderer Data asset, you'll need to add the feature once: select the Renderer Data asset in your Project window, open its Inspector, click **Add Renderer Feature -> CVD Renderer Feature**, then drag the `CvDSimulation` and `GaussianBlur` shaders (from `Runtime/Shaders/`) into the **Cvd Shader** and **Blur Shader** fields.
3. Click **Toggle CVD Simulation** in the tool window to cycle through None, Protanopia, Deuteranopia, and Tritanopia.
4. Click **Toggle Low Vision Blur** to enable or disable the blur pass independently.

Finding your Renderer Data asset: your URP Pipeline Asset (the one referenced in Graphics settings) has a "Renderer List" in its Inspector. Click the entry there to jump to the actual Renderer Data asset, which is what this tool needs.

Contrast Meter

1. Enter **Play Mode**.
2. Click **Toggle Contrast Meter** in the tool window.
3. Click any UI element on screen. A badge appears showing the live WCAG contrast ratio and PASS/FAIL against the 4.5:1 (normal text) threshold, along with what background it detected.
4. Contrast Meter composites semi-transparent or overlapping UI layers back-to-front rather than reading a single layer in isolation, and falls back to the camera's background colour when an element has no ancestor background graphic.

Report Writer

1. Assign an **AccessibilityThresholds** asset in the tool window. A default one, `DefaultAccessibilityThresholds`, is included, or create your own via **Assets -> Create -> Accessibility Tester -> Thresholds** to set custom contrast and font-size minimums.
2. Click **Generate Report**. This scans every Canvas in the currently open scene for `Text` / `TextMeshProUGUI` elements, checks contrast and font size, and writes a timestamped JSON report to `<project root>/AccessibilityReports/`.
3. The "Scan in Play Mode" checkbox handles scenes where UI backgrounds depend on runtime camera behaviour: a parallax background, say, or a camera that only reaches its correct position once gameplay starts. Check the box, and the tool enters Play Mode, waits for the scene to settle, scans, then returns to Edit Mode automatically. One real limit here: that automated wait can only settle on whatever screen your game happens to reach on its own (often a title screen). It can't navigate menus for you. For scenes reached only after specific player input, get your game into that state manually first, then click Generate Report while genuinely in Play Mode; the tool detects this and skips the wait cycle.

4. Each element in the report includes a `backgroundConfidence` field explaining how its background colour was determined. `High` means it was read directly or rendered accurately; `Estimated` means it was approximated from a texture; `Low` means a fallback was used that may not be fully accurate, with the reason stated. Check this field before treating a borderline result as final.

Font Scaler

1. Set the **Scale Factor** slider (0.5x to 3x).
2. Click **Apply Font Scale**. All TMP text in the scene scales by that factor, and any elements that now overlap are highlighted magenta.
3. Click **Revert Font Scale** to restore original sizes and colours.

Known limitations

- Shader Controller requires URP. In non-URP (Built-in RP) projects, this section of the tool window is replaced with an informational message; Contrast Meter, Report Writer, and Font Scaler are unaffected.
- Report Writer can't determine the true background colour behind text with no ancestor UI graphic, when that background is a coloured sprite read through a white tint multiplier and the sprite's texture isn't marked "Read/Write Enabled" in its import settings. This is disclosed in the report's `backgroundConfidence` field rather than silently guessed.
- "Scan in Play Mode" can't navigate your game's UI for you. It settles at whatever screen your game reaches on its own after entering Play Mode, commonly a title or start screen. To scan gameplay-specific UI states, play to that state manually first, then click Generate Report.
- Report Writer's Edit Mode scans render the scene's camera(s) to an offscreen texture to determine backgrounds. Multi-camera scenes, such as a layered background plus gameplay plus UI camera rig, are composited in depth order automatically, though this adds a small amount of scan time compared to a purely static colour lookup.

Non-URP projects

If URP isn't installed, or the `ACCESSIBILITY_TESTER_URP` scripting define isn't set in your project's Player Settings, the Shader Controller UI is hidden automatically and everything else works normally. No manual file deletion is needed, and no errors are thrown.

If you do have URP installed and want Shader Controller available, add `ACCESSIBILITY_TESTER_URP` under **Edit -> Project Settings -> Player -> Other Settings -> Scripting Define Symbols**.

Running the tests

If you cloned the repository rather than importing the `.unitypackage` (which excludes tests):

1. **Window -> General -> Test Runner.**
2. **EditMode** tab, **Run All.** Covers WCAG contrast math and CVD matrix accuracy, verified against Machado, Oliveira & Fernandes 2009's published reference data.
3. **PlayMode** tab, **Run All.** Covers GPU-readback colour accuracy for the CVD shader.

License

MIT. See [LICENSE](#). A copy is also bundled inside `Runtime/LICENSE.txt` so it travels with the exported `.unitypackage`.